



The Superior University, Lahore

LAB-TASK (12)

Course Title:	Programming for AI				Course Code:	-	Credit Hours	1
Instructor:	Sir rasikh				Program:	BSDS		
Semester:	4th	Batch:	Morning	Section:	4A	Date:	-	
Time Allowed:	-				Maximum Marks:		N/A	
Student’s Name:	Shumaila Maryam				Roll No.	SU92-BSDSM-S25-062		

Work Assigned:

Task 1; 1. Run the HadithBot.ipynb file (as discussed in class). 2. Then create your own QnA Bot from the topic you selected in Lab 10, using the same pipeline: o Preprocess your dataset (text-based QnA data) o Embed questions/answers using Hugging Face MiniLM o Store vectors using FAISS o Search and match using similarity o Display results via a Flask + HTML UI



1. Introduction

FindMate is a simple campus lost and found QnA bot. The main purpose of this project is to help students ask questions about lost items, found items, campus contact details, common checking locations, and safety rules. In this version, the bot does not only use fixed keyword rules. It follows the same pipeline as the HadithBot lab task, where text is cleaned, converted into embeddings, stored in FAISS, and searched by similarity.

I selected this topic because students commonly lose things like ID cards, wallets, chargers, USBs, books, keys, bags, phones, earbuds, and bottles. A small QnA bot can guide them quickly without searching manually or asking many people.

2. Objectives

- To make a small and useful web application for campus lost and found support.
- To guide students about where they should report or check lost items.
- To provide contact information of Student Affairs and Security Office.
- To use Flask as a backend framework for handling web requests.
- To store replies and campus information separately in a JSON file.
- To design a clean and user-friendly chat interface using HTML and CSS.

3. Tools and Technologies Used

Python - main programming language.

Pandas - reading and handling CSV dataset.

Regex - preprocessing and cleaning text.

Sentence Transformers - loading Hugging Face MiniLM model.

Hugging Face MiniLM - converts text into embeddings.

FAISS - stores embeddings and performs similarity search.

Flask - backend web framework.

5. System Requirements

- Python should be installed on the system.
- Flask should be installed using requirements.txt.
- A browser such as Chrome, Edge, or Firefox is required to open the website.
- The application runs locally on port 5000.

6. Working

The working is simple and easy to understand:

1. The user opens the FindMate website in the browser.
2. The user types a message or clicks a quick action button.

3. JavaScript sends this message to the Flask route /chat using a POST request.
4. Flask receives the message and passes it to the make_reply() function.
5. The program checks keywords in the message.
6. According to the matched keywords, the reply is selected from data.json.
7. The reply is returned to the browser and shown in the chat box.

7. Explanation of Main Files

7.1 app.py

This is the main backend file of the project. It starts the Flask application, loads data from data.json, checks the user message, and returns a response. It contains routes for the homepage and chat API.

- The / route displays the main HTML page.
- The /chat route receives the user message and sends back a reply in JSON format.
- The make_reply() function contains the main decision-making logic.
- The find_item_type() function identifies the item category such as wallet, charger, book, keys, USB, or ID card.

7.2 index.html

This file creates the layout of the FindMate web page. It contains the sidebar, quick action buttons, chat area, message input field, and JavaScript functions. The frontend sends the message to the backend through fetch().

- Quick action buttons help users send common messages easily.
- The chat box displays user and application messages.
- JavaScript handles sending, receiving, and clearing messages.

7.3 style.css

This file is used for the design of the project. It gives the application a modern look with gradient background, cards, buttons, responsive layout, and chat message styling.

- It styles the sidebar and chat card.
- It defines button hover effects and message bubbles.
- It includes responsive design for mobile and tablet screens.

7.4 data.json

This file stores all the data used by the application. The benefit of using JSON is that replies and contact details can be changed without editing the Python code.

- Campus name stored in JSON: Superior University Gold Campus
- Contact details are stored for Student Affairs and Security Office.
- Common locations and lost/found item replies are also stored here.

7.5 requirements.txt

This file contains the Python packages required to run the project. In this project, Flask and Werkzeug are required.

8. Important Functions

Function	Description
get_data()	Loads project data from the data.json file.
check_words()	Checks whether any keyword exists in the user message.
contact_info()	Returns campus lost and found contact information.
places_info()	Returns common places where students can check lost items.
find_item_type()	Finds item type such as wallet, book, charger, USB, keys, or ID card.
make_reply()	Main function that decides what reply should be given to the user.
home()	Loads the main index.html page.
chat()	Receives message from frontend and returns the reply in JSON format.

9. Important Code

Loading JSON Data

```
def get_data():
    current_folder = os.path.dirname(os.path.abspath(__file__))
    full_path = os.path.join(current_folder, file_name)

    with open(full_path, "r", encoding="utf-8") as f:
        all_data = json.load(f)

    return all_data

lost_found_data = get_data()
```

This function finds the data.json file from the project folder and loads it into the program.

Checking Keywords

```
def check_words(user_text, words_list):
    text = user_text.lower()

    for word in words_list:
        if word.lower() in text:
            return True

    return False
```

This function is used many times in the project. It checks if the user message contains any expected keyword.

Chat Route

```
2
3 @app.route("/chat", methods=["POST"])
4 def chat():
5     try:
6         user_data = request.get_json()
7
8         if not user_data or "message" not in user_data:
9             return jsonify({"error": "Invalid request format"}), 400
10
11         user_msg = user_data["message"]
12         answer = make_reply(user_msg)
13
14         return jsonify({"reply": answer})
15
16     except Exception as e:
17         print("Some error occured:", e)
18         return jsonify({"error": "An error occurred while processing your message"}), 500
19
```

This route receives messages from the frontend. It returns the final reply as JSON so JavaScript can display it in the chat box.

10. Setup and Run Instructions

To run this project on a local system, follow these steps:

1. Open the project folder in VS Code or any editor.
2. Make sure app.py, data.json, requirements.txt, templates/index.html, and static/style.css are placed correctly.
3. Open terminal in the project folder.
4. Install required packages using: `pip install -r requirements.txt`
5. Run the application using: `python app.py`
6. Open the browser and visit: `http://127.0.0.1:5000`

11. Testing

I tested this by sending different types of messages and checking if the correct reply is returned.

Test Message	Expected Result	Status
Hi	Greeting message should be shown.	Pass
I lost my ID card	ID card lost instructions should be shown.	Pass
I found a wallet	Wallet found instructions should be shown.	Pass
Contact office	Office contact details should be	Pass

	shown.	
Common locations	Common checking places should be shown.	Pass
	Empty message warning should be returned.	Pass

12. Limitations

- This uses keyword matching, so it may not understand every sentence.
- It does not save chat history in a database.
- It does not have a login system for students or staff.
- It does not upload pictures of lost or found items.
- It only gives guidance and does not track real lost items.

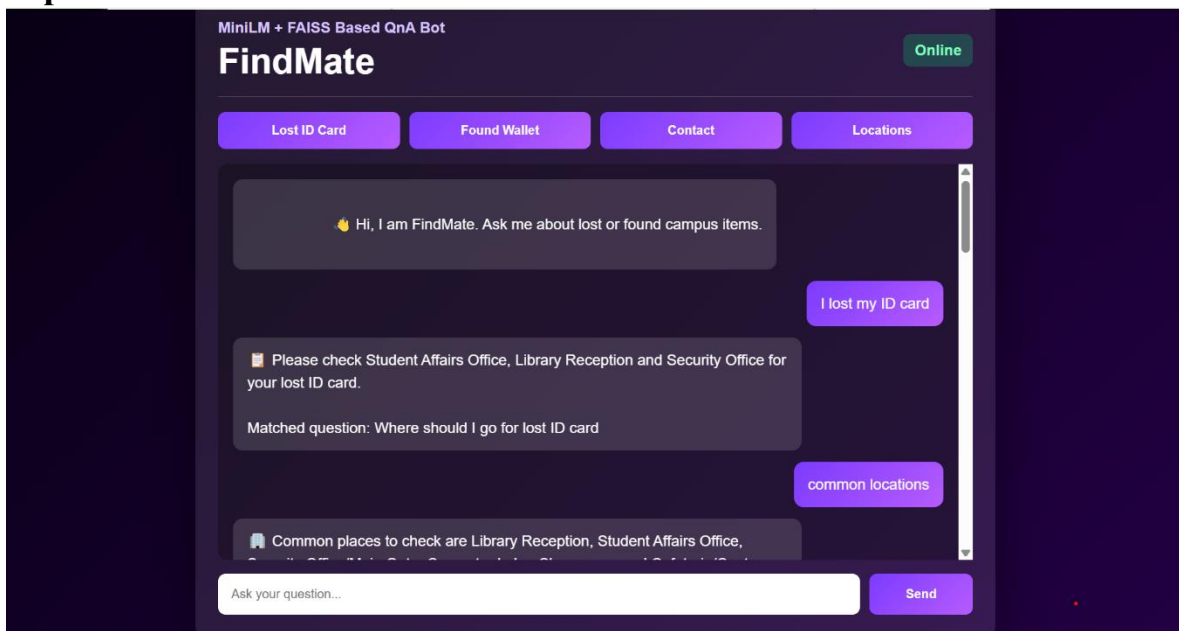
13. Future Enhancements

- A database can be added to store lost and found item reports.
- Admin panel can be added for Security Office or Student Affairs.
- Image upload option can be added for item proof.
- Search option can be added to find already reported items.
- Student login can be added for better tracking.
- Email notification can be added when a matching item is found.

14. Conclusion

FindMate is a simple but useful campus lost and found assistant. It helps students by giving quick guidance about lost and found items. The project uses Flask for backend, HTML/CSS/JavaScript for frontend, and JSON for storing replies. It is easy to understand and suitable for a semester project because it shows the basic working of a web application with frontend and backend communication.

Output:



Response:

Matched question: Common locations to check

I lost my ID card

 Please check Student Affairs Office, Library Reception and Security Office for your lost ID card.

Matched question: Where should I go for lost ID card

Ask your question...

Send